AQI Predictor — Project Write-Up

10 Pearls Data Science Internship ("Shine")

1. Overview

This project is a fully serverless, end-to-end machine learning system that forecasts Air Quality Index (AQI) three days ahead for three Pakistani cities — **Bahawalpur**, **Lahore**, and **Islamabad**. It collects hourly air-quality and weather data automatically, forecasts the daily-average AQI for the next three days, explains every prediction with SHAP, retrains itself daily without manual intervention, and presents everything through a public interactive dashboard.

Live dashboard: https://aqi-predictor-nslt99a5awaqkn4fypvqen.streamlit.app **Repository:** https://github.com/UshbaJ/aqi-predictor

Stack: Python, scikit-learn (Ridge, Random Forest), Hopsworks Serverless (feature store), GitHub Actions (automation/CI-CD), Streamlit Community Cloud (dashboard), SHAP (explainability), Git.

2. Data Collection

Sources:

- **OpenWeather Air Pollution API** — hourly pollutant concentrations (PM2.5, PM10, CO, NO, NO_2, O_3, SO_2, NH_3), used to compute the standard **US EPA AQI (0–500 scale)** from the dominant pollutant, rather than relying on OpenWeather's own coarse 1–5 index.
- **Open-Meteo Archive API** — historical weather (temperature, humidity, pressure, wind speed/direction), chosen over OpenWeather's paid historical weather endpoint to avoid any billing risk on a free-tier project.

Historical depth: Data was backfilled to OpenWeather's earliest available date (2020-11-27) for all three cities, pulled in yearly chunks to avoid API timeouts on a multi-year request — roughly **42,800–43,200 matched hourly rows per city**.

Data quality issue found and fixed: An early duplicate-timestamp bug was traced to timestamps not being floored to the hour before deduplication; fixed permanently by adding `.dt.floor("h")` to every save function, with a one-time cleanup script used to repair already-collected data.

3. Exploratory Data Analysis

EDA was performed per city (`notebooks/01_eda.ipynb`) and covers, for each of Bahawalpur, Lahore, and Islamabad:

- AQI over time, annotated with EPA "Unhealthy" (150) and "Very Unhealthy" (200) thresholds
- Pollutant correlation heatmaps
- Average AQI by hour of day (diurnal pollution pattern)

Key finding — temperature/pressure collinearity: Temperature and atmospheric pressure are strongly negatively correlated ($r = -0.80$) across the dataset. This single finding explains a downstream puzzle: SHAP

(computed on the Ridge model) and Random Forest's built-in feature importance disagree on which of the two features ranks higher at longer horizons — both are, in effect, encoding the same underlying weather-system signal, just weighting it differently depending on model architecture.

Cross-city comparison (added once all three cities had full historical data) shows Lahore consistently registers the highest average AQI across nearly all hours of the day, with Islamabad and Bahawalpur showing more variable relative rankings depending on time of day — supporting the decision to model each city independently rather than pooling data.

4. Feature Engineering

Implemented in `src/features.py`, parameterized by `city` throughout:

- **Lag features:** AQI at 1h, 3h, 6h, 12h, 24h prior; temperature/humidity/wind speed at 1h and 3h prior
- **Rolling statistics:** 6-hour rolling mean and standard deviation of AQI
- **Time features:** hour of day, day of week
- **Targets:** originally point-in-time AQI at +24h/+48h/+72h; **redesigned mid-project** to be **non-overlapping 24-hour window averages** — `target_24h` = mean AQI over hours [t+1, t+24], `target_48h` over [t+25, t+48], `target_72h` over [t+49, t+72]. This was the single largest structural change made to the modeling approach, triggered by confirming the correct specification partway through development.

5. Model Development and Validation

Models compared: naive persistence baseline (predict "no change"), Ridge regression, Random Forest.

Validation methodology: 5-fold `TimeSeriesSplit` cross-validation — never trains on future data to predict the past, and reports mean ± standard deviation across folds rather than a single train/test split, which would otherwise give a misleadingly precise (or volatile) number depending on which period landed in the test set.

Iteration history:

- An early validation attempt using only 30 days of matched AQI+weather data showed the naive baseline beating both models at 24h, and Ridge only marginally beating naive at 72h with negative R^2 throughout. Testing Ridge regularization strength (alpha 1.0 vs. 10.0) showed negligible difference, ruling out overfitting as the cause and correctly diagnosing it as a **data volume problem**.
- After the full 5-year historical pull (~42,840 rows), results improved substantially: 24h R^2 rose from 0.729 (naive) to 0.744 (Ridge); 72h R^2 rose from 0.365 (naive) to 0.551 (Random Forest) / 0.538 (Ridge) — confirming that weather features plus adequate data volume were both necessary, with the effect most pronounced at the longer 72h horizon.
- After the point-in-time → daily-average target redesign, final cross-validated results (Bahawalpur) were:

Horizon	Naive RMSE	Ridge RMSE	RF RMSE	Ridge improvement over naive
+24h	35.18 ± 5.33	28.72 ± 3.89	32.74 ± 7.53	18.4%
+48h	49.30 ± 7.20	39.97 ± 6.30	45.80 ± 11.59	18.9%
+72h	55.54 ± 9.12	44.14 ± 6.85	48.05 ± 9.06	20.5%

Ridge outperformed both naive and Random Forest at every horizon, with consistently lower variance across folds — the deciding factor in selecting it as the deployed model.

Genuine holdout validation (`validate_holdout.py`): the final deployed model is retrained excluding the most recent 90 days entirely, then evaluated only on that held-out window, demonstrating real forecasting performance rather than in-sample recall. Holdout RMSE for Bahawalpur's 24h model is ±14.19, consistent with the CV estimate.

Explainability (SHAP): `compute_shap.py` uses `shap.LinearExplainer` on the deployed Ridge models. For Bahawalpur, `aqi_epa` and `aqi_roll_mean_6h` dominate the 24h forecast, while `pressure` becomes the top feature at 72h. For Lahore and Islamabad, **temperature** (not AQI history) is the top SHAP feature at every horizon — a genuine cross-city difference, potentially reflecting different dominant pollution sources or seasonal dynamics in those cities.

6. MLOps and Automation

Feature Store: Hopsworks Serverless hosts one Feature Group per city (`aqi_weather_features` for Bahawalpur, `aqi_weather_features_{city}` for Lahore/Islamabad), version 2, HUDI format, offline-only.

CI/CD (GitHub Actions):

- **Hourly Feature Pipeline** — pulls the last 48 hours of data and pushes it to each city's Feature Group, as a **matrix job across all three cities in parallel** (`fail-fast: false`, so one city's transient failure doesn't cancel the others).
- **Daily Retraining** — retrains Ridge + Random Forest for all three horizons, per city, also matrixed in parallel, with trained models and CV results uploaded as workflow artifacts.

Reliability engineering — the Daily Retraining crash: The retraining workflow failed identically on three separate occasions with a two-stage failure: Hopsworks' Arrow Flight service intermittently drops its connection mid-read, and the code's local-CSV fallback then crashed with an unhandled `FileNotFoundError`, since `data/` is gitignored and never exists on a fresh GitHub Actions runner. This was root-caused and fixed by:

1. Adding retry-with-backoff around the Hopsworks read (most transient drops now self-heal)
2. Wrapping the fallback in its own exception handling, raising a purpose-built `FeatureLoadError` when both Hopsworks and local data are unavailable
3. Having `train.py` catch that error and exit cleanly (skip today's retrain, keep the existing model) instead of crashing with a raw traceback

Verified by mocking a Hopsworks failure to confirm the clean warning-and-skip behavior, then confirmed in real CI once deployed.

Other CI issues resolved: a separate minimal `requirements-pipeline.txt` was needed because the full dashboard requirements have a `streamlit`-vs-`hopsworks protobuf` version conflict that only matters for the dashboard, not the pipeline; a missing `pyarrow` dependency; and a dynamic `align_dtypes_to_schema()` function in `feature_pipeline.py` to match Hopsworks' locked-in per-column schema regardless of batch size or NaN presence.

7. Multi-City Expansion

After the single-city (Bahawalpur) pipeline was fully validated, stable, and automated, the same pipeline was extended to Lahore and Islamabad. This required:

- Parameterizing every script (`data_collection.py`, `features.py`, `feature_pipeline.py`, `train.py`, `predict.py`, `validate_holdout.py`, `compute_shap.py`, `app.py`) with a `city` argument, defaulting to `bahawalpur` for backward compatibility where appropriate
- A consistent naming convention across Hopsworks feature groups, model files, and output CSVs: Bahawalpur keeps its original unsuffixed names; Lahore and Islamabad use a `_{city}` suffix
- Migrating raw data storage from flat, inconsistently-named CSVs to a uniform per-city subfolder structure (`data/{city}/raw_aqi_data.csv`)
- Extending both GitHub Actions workflows to a `matrix` strategy running all three cities in parallel
- A city selector dropdown in the dashboard, threaded through every cached data-loading function so Streamlit correctly caches and refreshes per city

Several genuine bugs were caught and fixed during this expansion, including a `.gitignore` pattern that only matched files one directory deep (silently breaking after the subfolder migration), a Hopsworks `append_features()` crash specific to a freshly created (empty-schema) feature group, and city data files that existed as near-empty stubs rather than the real historical pulls, caught via a shape check before they could silently corrupt training.

8. Dashboard

Built with Streamlit, deployed to Streamlit Community Cloud. Organized as a persistent header and city selector, a current-conditions gauge and pollutant breakdown, a 3-day forecast with a hazard alert banner for any "Unhealthy" or worse day, and three tabs:

- **Forecast Details** — a what-if simulator (adjust temperature/humidity/pressure/wind/current AQI and see the model's live re-prediction), a text-to-speech voice briefing, a personal outdoor-exposure risk calculator, and the recent 14-day trend
- **Model & Validation** — per-city model performance vs. baseline, the 90-day holdout chart, and SHAP feature importance by horizon
- **City Comparison** — current AQI across all three cities, overlaid 14-day trends, an overlaid hourly-pollution-pattern chart, and full per-city EDA (time series, correlation heatmap, hourly pattern)

Deployment challenges resolved: Streamlit Cloud initially defaulted to Python 3.14, incompatible with `hopsworks==5.0.5` — fixed with an explicit `runtime.txt`. The full `requirements.txt` had a conflicting `sqlalchemy` pin and the same `protobuf` conflict as the CI pipeline — resolved by trimming it to only the packages actually needed at runtime. A separate cosmetic issue had visiting browsers with dark mode enabled forcing Streamlit's native dropdown into an unreadable dark style regardless of the app's own light theme — root-caused to Streamlit's own theme engine detecting browser preference, and fixed with an explicit `.streamlit/config.toml` forcing `base = "light"`.

9. Findings Summary

- Weather features materially improve AQI forecasts, but only once sufficient historical data volume is available — the effect was invisible on 30 days of data and clear on 5 years.
- Ridge regression, despite its simplicity, consistently outperformed Random Forest at every horizon and every city, with lower variance.

- Temperature and pressure's strong negative correlation is the single most important structural fact about this dataset, explaining an otherwise-confusing disagreement between two different feature-importance methods.
- Newer cities (Lahore, Islamabad) show materially higher forecast variance at longer horizons than Bahawalpur, consistent with their shorter effective production history.
- Cross-city SHAP importance differs meaningfully: Bahawalpur's forecasts are driven primarily by AQI history, while Lahore's and Islamabad's are driven primarily by temperature.